CS3014 - Estructuras de Datos Avanzadas

Enteros II
Semana 11

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

23 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Ver la vEB como un árbol binario sobre los bits y buscar en él con una búsqueda binaria sobre la profundidad
- Construir los X-fast trees: tabla hash de prefijos, $O(\lg w)$ por consulta

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Ver la vEB como un árbol binario sobre los bits y buscar en él con una búsqueda binaria sobre la profundidad
- Construir los X-fast trees: tabla hash de prefijos, $O(\lg w)$ por consulta
- Reducir el espacio y la actualización con indirección y llegar a los Y-fast trees: $O(\lg w)$ y $\Theta(n)$
- Demostrar la amortización de la indirección

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La deuda de la vEB

Con $w = 32$ una vEB reserva unas $4 \cdot 10^9$ palabras aunque guarde 10 llaves, y con $w = 64$ ni siquiera cabe en memoria.

¿Se puede conservar el tiempo $O(\lg w)$ y pagar solo por las n llaves?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La deuda de la vEB

Con $w = 32$ una vEB reserva unas $4 \cdot 10^9$ palabras aunque guarde 10 llaves, y con $w = 64$ ni siquiera cabe en memoria.

¿Se puede conservar el tiempo $O(\lg w)$ y pagar solo por las n llaves?

Sí, con dos ideas

Guardar **solo lo que existe** (una tabla hash) y agrupar las llaves en bloques de tamaño $\Theta(w)$ para actualizar la estructura con poca frecuencia.

Ahorrar espacio en la vEB

No guardar los clusters vacíos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla hash en lugar de un arreglo de clusters

$V.cluster[c]$ pasa a ser una entrada de una tabla hash que solo existe cuando el cluster es no vacío. Con hashing perfecto dinámico, buscar y crear una entrada cuestan $O(1)$ esperado.

No guardar los clusters vacíos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla hash en lugar de un arreglo de clusters

$V.cluster[c]$ pasa a ser una entrada de una tabla hash que solo existe cuando el cluster es no vacío. Con hashing perfecto dinámico, buscar y crear una entrada cuestan $O(1)$ esperado.

¿Cuánto espacio ocupa ahora?

No guardar los clusters vacíos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una tabla hash en lugar de un arreglo de clusters

$V.cluster[c]$ pasa a ser una entrada de una tabla hash que solo existe cuando el cluster es no vacío. Con hashing perfecto dinámico, buscar y crear una entrada cuestan $O(1)$ esperado.

¿Cuánto espacio ocupa ahora?

$O(n \lg w)$

Cada inserción hace una sola llamada costosa por nivel de recursión y hay $\lg w$ niveles: cada llave deja $O(1)$ registros por nivel. Con $w = 32$ y 1024 llaves medimos entre 3,3 y 3,9 registros por llave.

No guardar los clusters vacíos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Se puede llegar a $\Theta(n)$ siguiendo por este camino?

No guardar los clusters vacíos

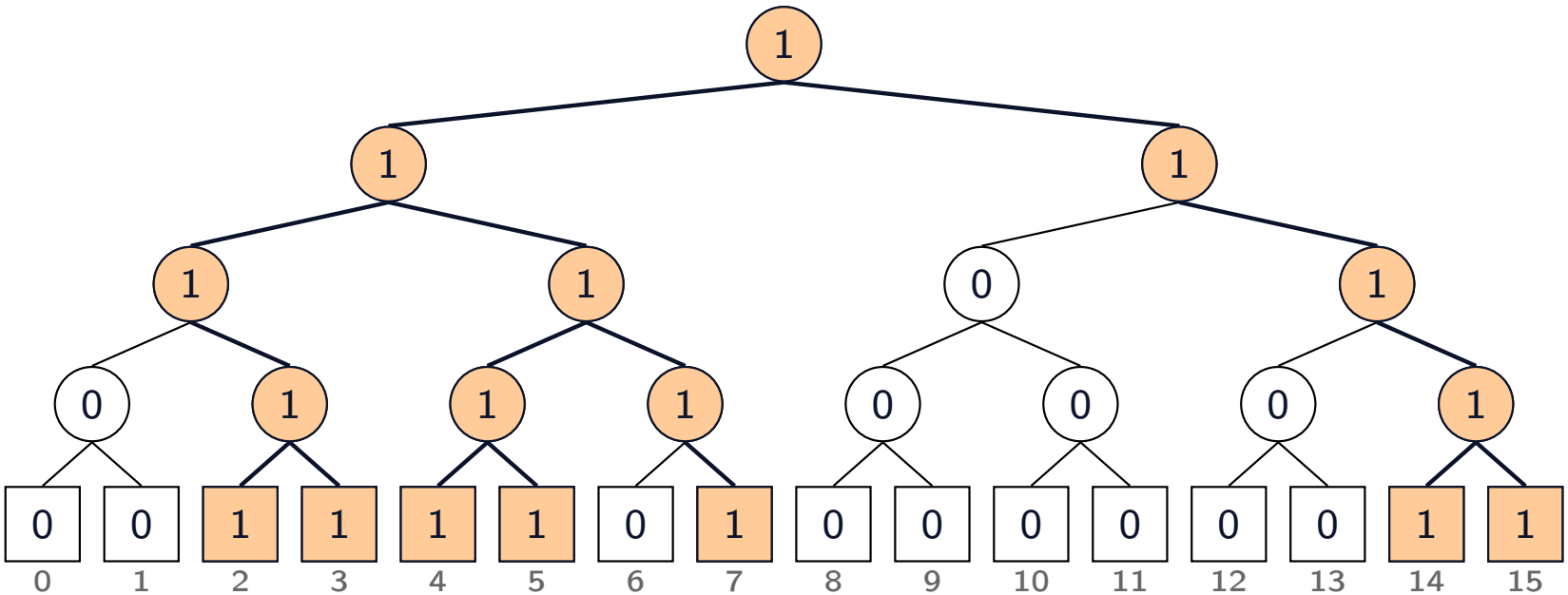
¿Se puede llegar a $\Theta(n)$ siguiendo por este camino?

Con refinamientos más complejos (Mehlhorn y Näher), sí

Nosotros tomamos otro camino: reducir el problema a una estructura más simple y luego aplicar **indirección**. Para eso hace falta mirar la vEB de otra forma.

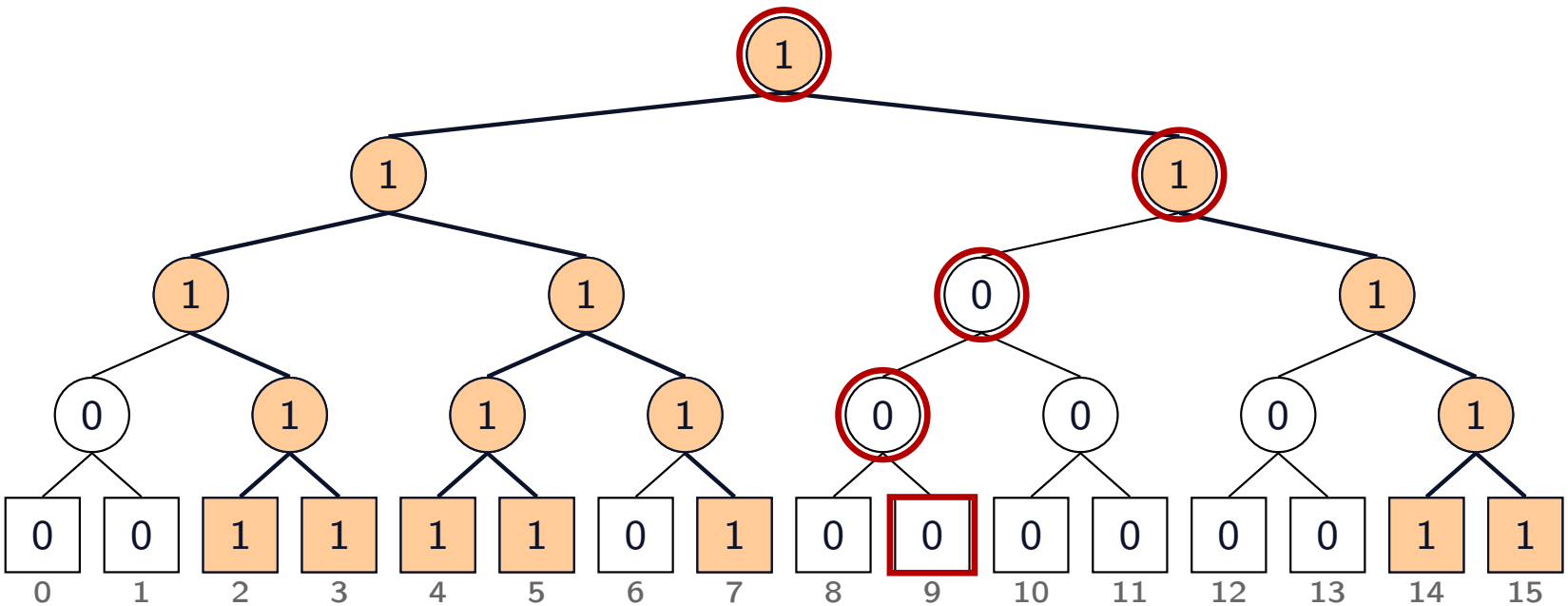
La vista de árbol

El conjunto como un árbol binario



Un árbol binario perfecto de altura w sobre los u bits ($u = 16$, $w = 4$). Cada nodo vale el OR de sus hijos: 1 si su intervalo contiene alguna llave. El nodo a profundidad ℓ en el camino de x corresponde a sus ℓ bits más altos: $x \gg (w - \ell)$.

La ruta hacia x



Ruta de $x = 9 = 1001_2$: valores $1, 1, 0, 0, 0$

Si un nodo vale 0, todos sus descendientes valen 0: a lo largo de la ruta los valores son $1 \cdots 10 \cdots 0$.

Buscar la transición $1 \rightarrow 0$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cómo encontrar el último nodo con valor 1 en la ruta?

Buscar la transición $1 \to 0$

¿Cómo encontrar el último nodo con valor 1 en la ruta?

Con una búsqueda binaria sobre la profundidad $\ell \in [0, w]$

Se necesita el valor del nodo a profundidad ℓ en $O(1)$: el nodo es el prefijo $x \gg (w - \ell)$, una operación aritmética de la word RAM. En $O(\lg w)$ probes se llega al último 1, el nodo p.

Buscar la transición $1 \to 0$

¿Cómo encontrar el último nodo con valor 1 en la ruta?

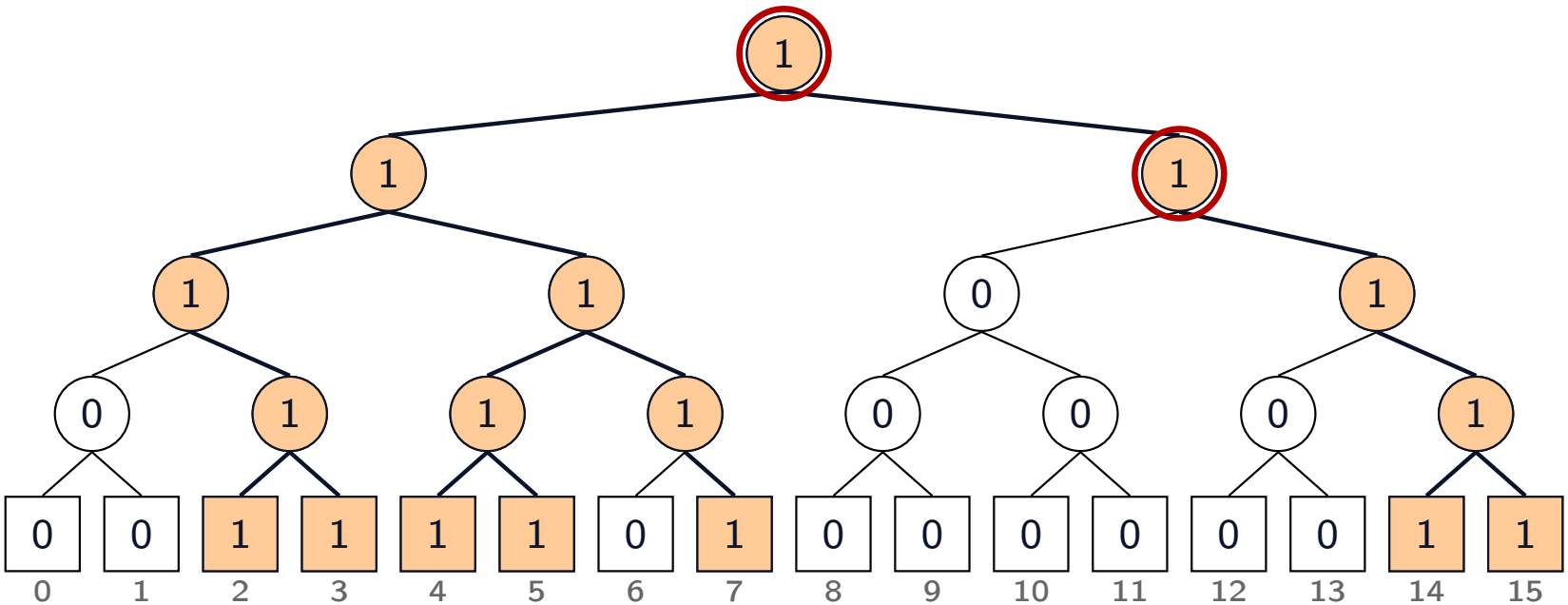
Con una búsqueda binaria sobre la profundidad $\ell \in [0, w]$

Se necesita el valor del nodo a profundidad ℓ en $O(1)$: el nodo es el prefijo $x \gg (w - \ell)$, una operación aritmética de la word RAM. En $O(\lg w)$ probes se llega al último 1, el nodo p.

Del nodo p al sucesor

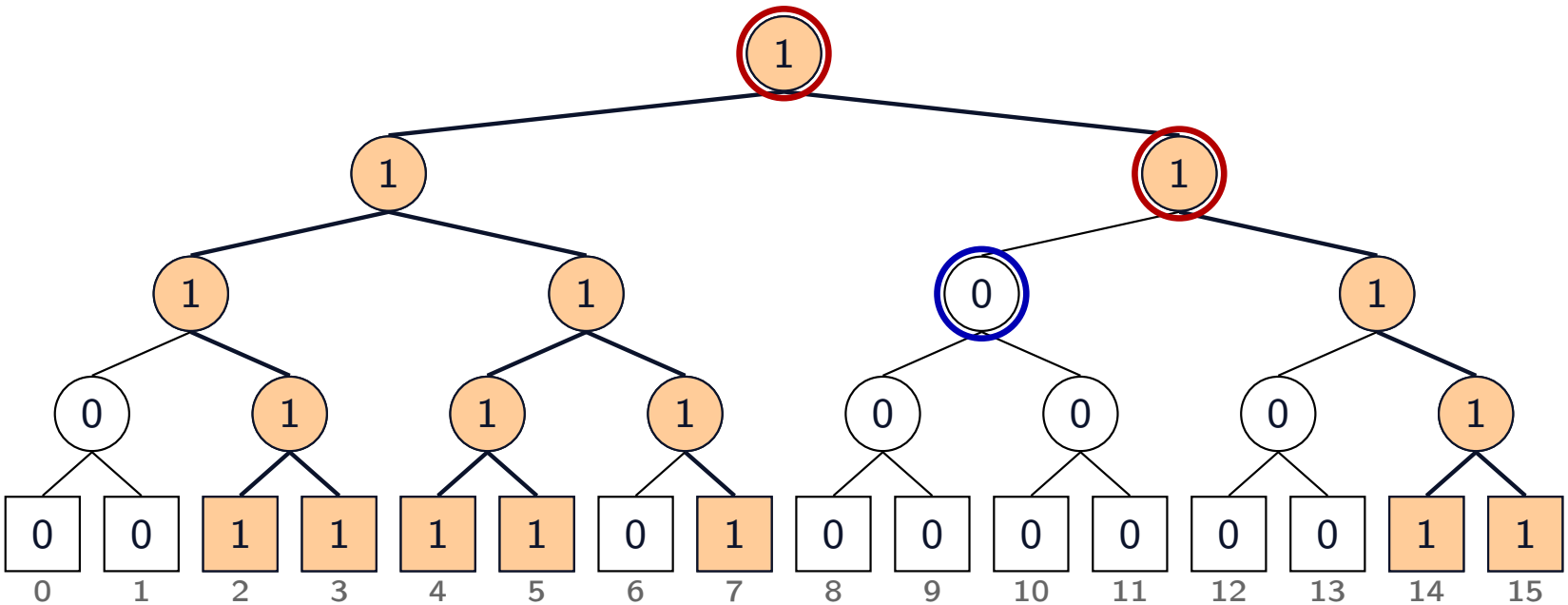
El hijo de p hacia x vale 0. Si es el hijo izquierdo, el sucesor es el **mínimo** del hijo derecho. Si es el derecho, el predecesor es el **máximo** del hijo izquierdo. Cada nodo guarda *min* y *max*.

Un ejemplo: Sucesor(9)



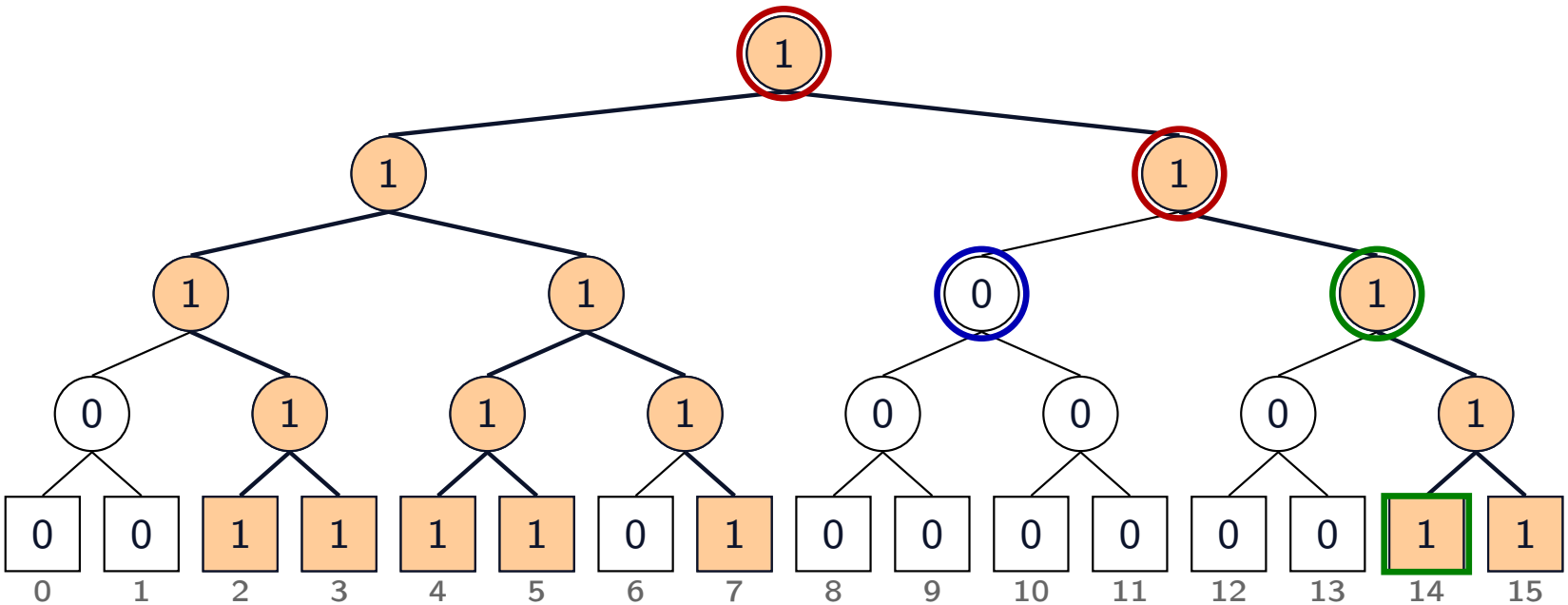
- El último 1 en la ruta de 9 es p = prefijo 1 (rojo).

Un ejemplo: Sucesor(9)



- El último 1 en la ruta de 9 es p = prefijo 1 (rojo).
- Su hijo hacia 9 es el prefijo 10 (azul), con valor 0: es el hijo izquierdo.

Un ejemplo: Sucesor(9)



- El último 1 en la ruta de 9 es p = prefijo 1 (rojo).
- Su hijo hacia 9 es el prefijo 10 (azul), con valor 0: es el hijo izquierdo.
- El sucesor es el mínimo del hijo derecho, el prefijo 11 (verde): 14.

¿Qué cuesta esta versión?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Consulta: $O(\lg w)$

Una búsqueda binaria sobre las $w + 1$ profundidades.

¿Qué cuesta esta versión?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Consulta: $O(\lg w)$

Una búsqueda binaria sobre las $w + 1$ profundidades.

¿Y la actualización y el espacio?

¿Qué cuesta esta versión?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Consulta: $O(\lg w)$

Una búsqueda binaria sobre las $w + 1$ profundidades.

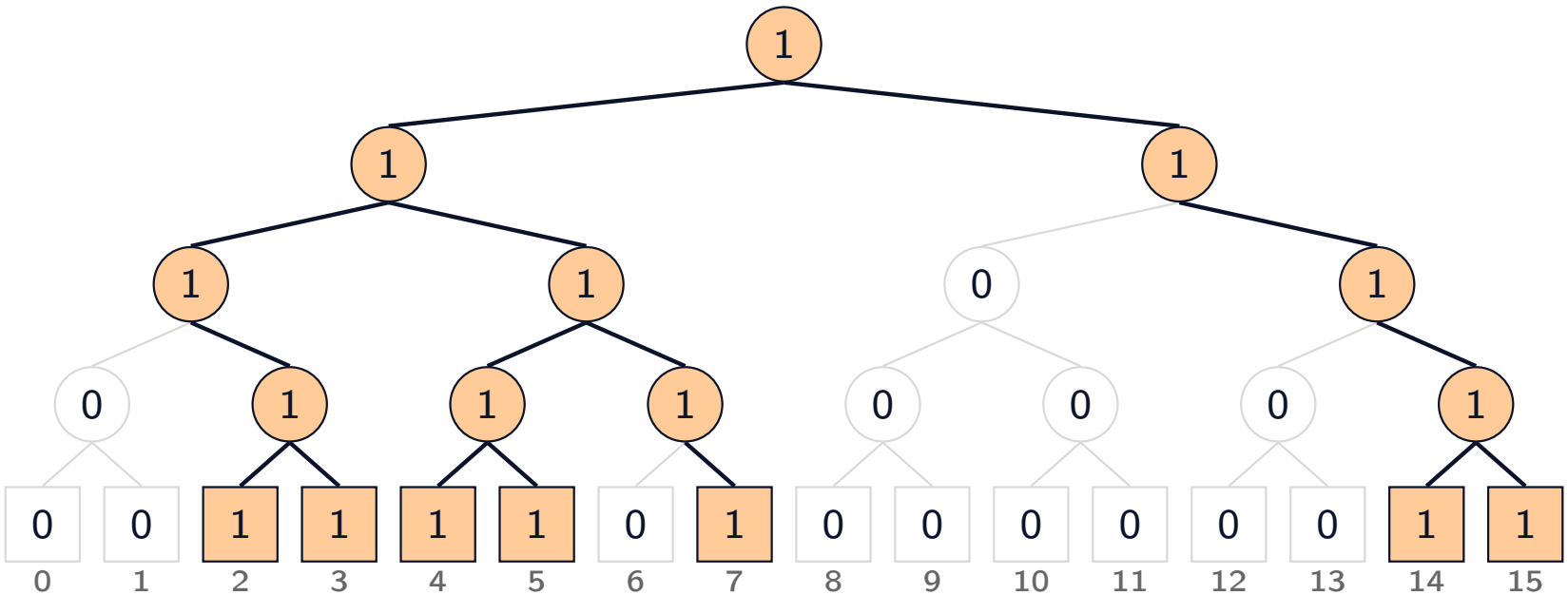
¿Y la actualización y el espacio?

Actualizar: $O(w)$. Espacio: $\Theta(u)$

Insertar x cambia los $w + 1$ nodos de su ruta (el OR, el *min* y el *max*), y el árbol tiene $2u - 1$ nodos. Es la misma consulta que la vEB, pero con actualización más cara.

X-fast trees

Guardar solo los nodos con valor 1



La tabla T

Una tabla hash con una entrada por cada prefijo de cada llave: la clave es el par $(\ell, \text{prefijo})$ y el valor es (min, max) del subárbol. Más una lista doblemente enlazada con las llaves en orden.

La tabla del ejemplo

¿Cuántas entradas tiene T para $S = \{2, 3, 4, 5, 7, 14, 15\}$?

La tabla del ejemplo

¿Cuántas entradas tiene T para $S = \{2, 3, 4, 5, 7, 14, 15\}$?

17: los nodos con valor 1 del dibujo

1 en la raíz, 2 a profundidad 1, 3 a profundidad 2, 4 a profundidad 3 y 7 hojas. Como mucho $n(w + 1)$ en general.

La tabla del ejemplo

¿Cuántas entradas tiene T para $S = \{2, 3, 4, 5, 7, 14, 15\}$?

17: los nodos con valor 1 del dibujo

1 en la raíz, 2 a profundidad 1, 3 a profundidad 2, 4 a profundidad 3 y 7 hojas. Como mucho $n(w + 1)$ en general.

$(\ell, \text{prefijo})$	(min, max)
$(0, \varepsilon)$	$(2, 15)$
$(1, 0)$	$(2, 7)$
$(2, 01)$	$(4, 7)$
$(3, 111)$	$(14, 15)$

Sucesor en un X-fast tree

Algoritmo 1: SUCESOR-X(x)

$lo \leftarrow 0,\ hi \leftarrow w$ // el prefijo de longitud 0 siempre está
mientras $lo < hi$ **hacer**
    $mid \leftarrow \lceil (lo + hi)/2 \rceil$
    si $(mid,\ x \gg (w - mid)) \in T$ **entonces**
        $lo \leftarrow mid$
    en otro caso
        $hi \leftarrow mid - 1$
$\ell \leftarrow lo,\ p \leftarrow x \gg (w - \ell)$
si $\ell = w$ **entonces devolver** $sig(x)$ // $x \in S$: el siguiente de la lista

si *el bit* $\ell + 1$ *de* x *es* 0 **entonces**
    devolver $T[(\ell + 1,\ 2p + 1)].min$ // falta el hijo izquierdo
en otro caso
    $y \leftarrow T[(\ell + 1,\ 2p)].max$ // falta el hijo derecho: y es el predecesor
    devolver $sig(y)$

Un ejemplo: Sucesor(6) y Sucesor(13)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$x = 6 = 0110_2$

Los prefijos de x en T son ε, 0, 01, 011; el de longitud 4 (0110) no está. Entonces $\ell = 3$, $p = 011$, y el bit siguiente de x es 0.

Un ejemplo: Sucesor(6) y Sucesor(13)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$x = 6 = 0110_2$

Los prefijos de x en T son ε, 0, 01, 011; el de longitud 4 (0110) no está. Entonces $\ell = 3$, $p = 011$, y el bit siguiente de x es 0.

- Falta el hijo izquierdo 0110; se devuelve el *min* del derecho, $T[(4, 0111)] = 7$.

Un ejemplo: Sucesor(6) y Sucesor(13)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$x = 6 = 0110_2$

Los prefijos de x en T son ε, 0, 01, 011; el de longitud 4 (0110) no está. Entonces $\ell = 3$, $p = 011$, y el bit siguiente de x es 0.

- Falta el hijo izquierdo 0110; se devuelve el *min* del derecho, $T[(4, 0111)] = 7$.

$x = 13 = 1101_2$

Prefijos presentes: ε, 1, 11. Entonces $\ell = 2$, $p = 11$ y el bit siguiente es 0 (prefijo 110 ausente).

Un ejemplo: Sucesor(6) y Sucesor(13)

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$x = 6 = 0110_2$

Los prefijos de x en T son ε, 0, 01, 011; el de longitud 4 (0110) no está. Entonces $\ell = 3$, $p = 011$, y el bit siguiente de x es 0.

- Falta el hijo izquierdo 0110; se devuelve el *min* del derecho, $T[(4, 0111)] = 7$.

$x = 13 = 1101_2$

Prefijos presentes: ε, 1, 11. Entonces $\ell = 2$, $p = 11$ y el bit siguiente es 0 (prefijo 110 ausente).

- Falta el hijo izquierdo 110: sucesor $= T[(3, 111)].min = 14$.

Insertar y eliminar

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Insertar(x)

Buscar el sucesor con Sucesor-X para ubicar a x en la lista y agregar los $w + 1$ prefijos de x a T, actualizando *min* y *max* de los que ya existían.

Eliminar(x)

En cada prefijo de x: si x era la única llave, se borra la entrada; si era el *min* (o el *max*), el nuevo valor es el siguiente (o el anterior) de x en la lista.

Insertar y eliminar

Insertar(x)

Buscar el sucesor con Sucesor-X para ubicar a x en la lista y agregar los $w+1$ prefijos de x a T, actualizando *min* y *max* de los que ya existían.

Eliminar(x)

En cada prefijo de x: si x era la única llave, se borra la entrada; si era el *min* (o el *max*), el nuevo valor es el siguiente (o el anterior) de x en la lista.

Tiempo $\Theta(w)$ por actualización

Es lo mismo que en el árbol sin hash: hay que tocar toda la ruta.

Resumen del X-fast tree

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

	Consulta	**Actualización**	**Espacio**
vEB	$O(\lg w)$	$O(\lg w)$	$\Theta(u)$
X-fast tree	$O(\lg w)$	$\Theta(w)$	$O(nw)$

Resumen del X-fast tree

	Consulta	**Actualización**	**Espacio**
vEB	$O(\lg w)$	$O(\lg w)$	$\Theta(u)$
X-fast tree	$O(\lg w)$	$\Theta(w)$	$O(nw)$

¿Cómo evitar pagar w por actualización y w de espacio por llave?

Resumen del X-fast tree

	Consulta	Actualización	Espacio
vEB	$O(\lg w)$	$O(\lg w)$	$\Theta(u)$
X-fast tree	$O(\lg w)$	$\Theta(w)$	$O(nw)$

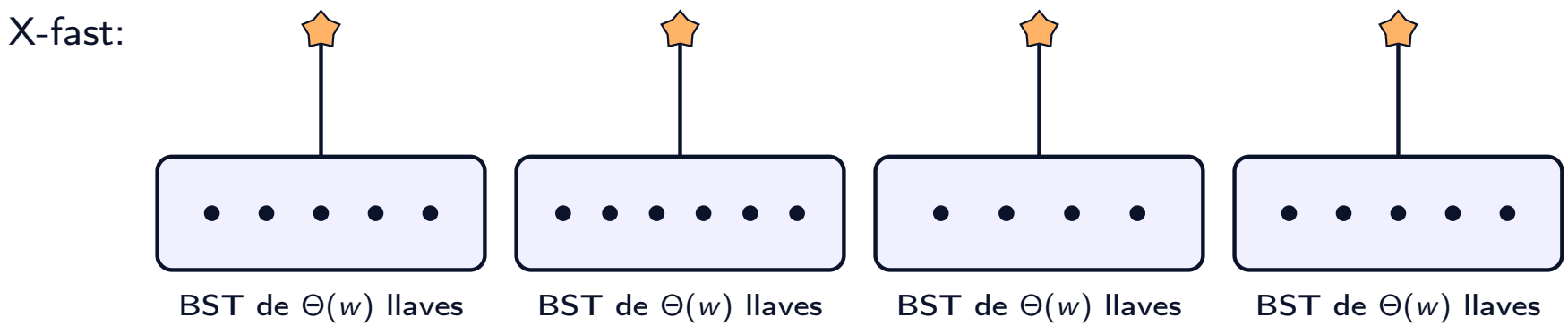
¿Cómo evitar pagar w por actualización y w de espacio por llave?

Haciendo que el X-fast tree guarde solo n/w llaves

Si cada llave del X-fast tree representa a $\Theta(w)$ llaves, su espacio total es $(n/w) \cdot w = n$.

Y-fast trees

Indirección



Las llaves ordenadas se parten en grupos contiguos de $\Theta(w)$; cada grupo vive en un BST y un **representante** entra al X-fast tree.

El invariante de tamaño

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Cada grupo tiene entre $w/2$ y $2w$ llaves

- Si crece a $2w + 1$: se parte en dos grupos de w y $w + 1$.
- Si baja de $w/2$: se fusiona con un vecino; si el resultado supera $1{,}5w$, se parte por la mitad.

El invariante de tamaño

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Cada grupo tiene entre $w/2$ y $2w$ llaves

- Si crece a $2w + 1$: se parte en dos grupos de w y $w + 1$.
- Si baja de $w/2$: se fusiona con un vecino; si el resultado supera $1{,}5w$, se parte por la mitad.

Los representantes

Cada grupo tiene un representante r_j fijado al crear el grupo (por ejemplo su mínimo en ese momento). Aunque la llave r_j se elimine, se queda en el X-fast tree hasta la próxima partición o fusión. Así $G_j = \{x \in S \mid r_j \leq x < r_{j+1}\}$.

Espacio

¿Cuántas llaves tiene el X-fast tree de arriba?

Espacio

¿Cuántas llaves tiene el X-fast tree de arriba?

A lo más $2n/w$: $O(n)$ en total

Cada grupo tiene al menos $w/2$ llaves, así que hay a lo más $2n/w$ representantes. Cada uno ocupa $O(w)$ en el X-fast tree: $O(n)$. Los BST suman $O(n)$.

Sucesor en un Y-fast tree

Algoritmo 2: SUCESOR-Y(x)

$r_j \leftarrow$ el mayor representante $\leq x$ (consulta al X-fast tree)
si *hay una llave* $> x$ *en el BST de* G_j **entonces devolver** la menor de ellas
si *existe* G_{j+1} **entonces devolver** el mínimo de G_{j+1}
devolver nulo

Sucesor en un Y-fast tree

Algoritmo 3: SUCESOR-Y(x)

$r_j \leftarrow$ el mayor representante $\leq x$ (consulta al X-fast tree)
si *hay una llave* $> x$ *en el BST de* G_j **entonces devolver** la menor de ellas
si *existe* G_{j+1} **entonces devolver** el mínimo de G_{j+1}
devolver nulo

$O(\lg w) + O(\lg w) = O(\lg w)$

La búsqueda de r_j es del X-fast tree; las demás búsquedas son en BST de $\Theta(w)$ llaves. El predecesor es simétrico: busca en G_j y, si no hay, toma el máximo de G_{j-1}.

¿Por qué solo G_j y G_{j+1}?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Si $r_j \leq x < r_{j+1}$, ¿puede el sucesor de x estar en G_{j+2} o más lejos?

¿Por qué solo G_j y G_{j+1}?

Si $r_j \leq x < r_{j+1}$, ¿puede el sucesor de x estar en G_{j+2} o más lejos?

No

Todas las llaves de G_{j+1} son $\geq r_{j+1} > x$. Si G_j no tiene llaves mayores que x, la menor llave mayor que x es el mínimo de G_{j+1}; y si G_{j+1} no existe, no hay sucesor.

Insertar y eliminar

Insertar(x)

Encontrar G_j con el X-fast tree ($O(\lg w)$), insertar en su BST ($O(\lg w)$). Si el grupo supera $2w$ llaves, partirlo.

Eliminar(x)

Eliminar de su BST. Si el grupo baja de $w/2$, fusionarlo con un vecino (y partir si hace falta).

Insertar y eliminar

Insertar(x)

Encontrar G_j con el X-fast tree ($O(\lg w)$), insertar en su BST ($O(\lg w)$). Si el grupo supera $2w$ llaves, partirlo.

Eliminar(x)

Eliminar de su BST. Si el grupo baja de $w/2$, fusionarlo con un vecino (y partir si hace falta).

Una partición o fusión cuesta $\Theta(w)$. ¿No arruina eso el tiempo?

Insertar y eliminar

UTEC
UNIVERSIDAD DE INGENIERÍA

Insertar(x)

Encontrar G_j con el X-fast tree ($O(\lg w)$), insertar en su BST ($O(\lg w)$). Si el grupo supera $2w$ llaves, partirlo.

Eliminar(x)

Eliminar de su BST. Si el grupo baja de $w/2$, fusionarlo con un vecino (y partir si hace falta).

Una partición o fusión cuesta $\Theta(w)$. ¿No arruina eso el tiempo?

Solo si ocurren seguido, y no ocurren seguido

Reconstruir los BST cuesta $O(w)$, y cambiar un representante en el X-fast tree cuesta $O(w)$. Veamos cada cuánto sucede.

Cada cuánto se reestructura

Tras una partición o fusión, ¿cuántas actualizaciones hacen falta en ese grupo para que haya otra?

Cada cuánto se reestructura

Tras una partición o fusión, ¿cuántas actualizaciones hacen falta en ese grupo para que haya otra?

Al menos $w/4$

- Partición de $2w+1$: grupos de tamaño w y $w+1$. Para volver a $2w+1$ hacen falta $\geq w$ inserciones; para bajar de $w/2$, $\geq w/2$ eliminaciones.
- Fusión: el resultado mide $\geq w-1$ y, si supera $1{,}5w$, se parte en grupos de $(0{,}75w,\ 1{,}25w)$: para llegar a $2w$ o a $w/2$ hacen falta $\geq w/4$ operaciones.

La cota amortizada

Teorema

INSERTAR y ELIMINAR cuestan $O(\lg w)$ amortizado.

La cota amortizada

Teorema

INSERTAR y ELIMINAR cuestan $O(\lg w)$ amortizado.

Demostración

Cada operación paga $O(\lg w)$ por la búsqueda en el X-fast tree y el BST. Una reestructuración cuesta $O(w)$ y ocurre a lo más una vez por cada $w/4$ operaciones de ese grupo: se cobra $O(w)/(w/4) = O(1)$ a cada una. Total: $O(\lg w) + O(1)$.

Resumen de las cuatro estructuras

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

	Consulta	**Actualización**	**Espacio**
vEB	$O(\lg w)$	$O(\lg w)$	$\Theta(u)$
vEB con hash	$O(\lg w)$	$O(\lg w)$	$O(n \lg w)$
X-fast tree	$O(\lg w)$	$\Theta(w)$	$O(nw)$
Y-fast tree	$O(\lg w)$	$O(\lg w)$ amort.	$\Theta(n)$

Resumen de las cuatro estructuras

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

	Consulta	**Actualización**	**Espacio**
vEB	$O(\lg w)$	$O(\lg w)$	$\Theta(u)$
vEB con hash	$O(\lg w)$	$O(\lg w)$	$O(n \lg w)$
X-fast tree	$O(\lg w)$	$\Theta(w)$	$O(nw)$
Y-fast tree	$O(\lg w)$	$O(\lg w)$ amort.	$\Theta(n)$

Con tablas hash, los tiempos de X-fast y Y-fast son esperados (o con alta probabilidad). Para $n = 10^9$ y $w = 64$: $\lg w = 6$ contra $\lg n \approx 30$.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La vEB como árbol binario: los valores a lo largo de la ruta de x son $1 \cdots 10 \cdots 0$, y una búsqueda binaria sobre la profundidad, con *min* y *max* en cada nodo, da el sucesor en $O(\lg w)$.
- X-fast: solo los nodos con 1, en una tabla hash de prefijos. Consulta $O(\lg w)$, pero actualización $\Theta(w)$ y espacio $O(nw)$.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- La vEB como árbol binario: los valores a lo largo de la ruta de x son $1 \cdots 10 \cdots 0$, y una búsqueda binaria sobre la profundidad, con *min* y *max* en cada nodo, da el sucesor en $O(\lg w)$.
- X-fast: solo los nodos con 1, en una tabla hash de prefijos. Consulta $O(\lg w)$, pero actualización $\Theta(w)$ y espacio $O(nw)$.
- Y-fast: indirección con grupos de $\Theta(w)$ en BST y un X-fast de representantes. Espacio $\Theta(n)$, consulta $O(\lg w)$.
- La actualización cuesta $O(\lg w)$ amortizado: reestructurar cuesta $O(w)$ y ocurre una vez cada $\Omega(w)$ operaciones.

Logros de la sesión



Logros de la sesión

- Sabemos buscar el sucesor en el árbol binario con una búsqueda binaria sobre la profundidad
- Sabemos construir un X-fast tree y ejecutar sus consultas

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos buscar el sucesor en el árbol binario con una búsqueda binaria sobre la profundidad
- Sabemos construir un X-fast tree y ejecutar sus consultas
- Sabemos aplicar indirección y justificar los tamaños de grupo
- Sabemos demostrar que un Y-fast tree usa $\Theta(n)$ de espacio y $O(\lg w)$ por operación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- van Emde Boas, P. (1975). Preserving order in a forest in less than logarithmic time. *Proceedings of FOCS'75*, 75–84.
- Willard, D. E. (1983). Log-logarithmic worst-case range queries are possible in space $\Theta(n)$. *Information Processing Letters*, 17(2), 81–84.
- Mehlhorn, K., & Näher, S. (1990). Bounded ordered dictionaries in $O(\log \log N)$ time and $O(n)$ space. *Information Processing Letters*, 35(4), 183–189.
- Demaine, Erik (2012). *6.851: Advanced Data Structures (Spring'12)*, Lecture 11. `https://courses.csail.mit.edu/6.851/spring12/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.